Practical No.6

Title: Image processing and recognitionfor color and shape detection
Aim: Shape detection using Opencv Python

Theory:
Introduction to Computer Vision:

Computer Vision is a field of Artificial Intelligence that enables computers to **see, interpret, and understand images and videos**, similar to how humans use their eyes and brain.

Goals of Computer Vision

- Recognize objects (cars, faces, animals)
- Understand scenes (indoor vs outdoor)
- Track motion (surveillance, sports analytics)
- Extract useful information from visuals

Real-world Applications

- Face recognition (used in smartphones)
- Self-driving cars
- Medical image analysis (X-rays, MRIs)
- Security surveillance
- Augmented reality (filters, games)

Introduction to OpenCV:

OpenCV is a popular open-source library used for **computer vision and image processing**.

- Written in C++, but widely used with Python
- Works on Windows, Linux, macOS
- Designed for **real-time applications**

Why Use OpenCV?

- Free and open-source

- Huge community support
- Fast and efficient
- Supports hundreds of functions for image/video processing

Key Features of OpenCV

1. Image Processing

- Read, write, display images
- Resize, crop, rotate

2. Video Processing

- Capture video from webcam
- Process frames in real time

3. Object Detection

- Face detection
- Pedestrian detection

4. Drawing Functions

- Draw lines, rectangles, circles on images

5. Image Transformations

- Blurring, sharpening
- Edge detection (like Canny)

Code:

```
import cv2

import numpy as np

from google.colab.patches import cv2_imshow   # ✅ important
```

```
# Read image
img = cv2.imread('/input.png')   # make sure file exists in Colab
img = cv2.resize(img, (600, 600))
output = img.copy()

print("Before Image Processing:")
cv2_imshow(output)
# Convert to HSV
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# -------- Color Ranges --------
lower_red1 = np.array([0, 120, 70])
upper_red1 = np.array([10, 255, 255])
lower_red2 = np.array([170, 120, 70])
upper_red2 = np.array([180, 255, 255])

lower_green = np.array([36, 50, 70])
upper_green = np.array([89, 255, 255])

lower_blue = np.array([90, 50, 70])
upper_blue = np.array([128, 255, 255])
```

```
lower_yellow = np.array([20, 100, 100])

upper_yellow = np.array([35, 255, 255])


# -------- Shape Detection --------

def detect_shape(cnt):

    peri = cv2.arcLength(cnt, True)

    approx = cv2.approxPolyDP(cnt, 0.04 * peri, True)


    if len(approx) == 3:

        return "Triangle"

    elif len(approx) == 4:

        return "Rectangle"

    else:

        return "Circle"


# -------- Process Function --------

def detect(mask, color_name):

    kernel = np.ones((5, 5), np.uint8)

    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)


    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)


    for cnt in contours:
```

```
        if cv2.contourArea(cnt) > 700:
            shape = detect_shape(cnt)
            x, y, w, h = cv2.boundingRect(cnt)

            cv2.drawContours(output, [cnt], -1, (0, 0, 0), 2)
            cv2.putText(output, color_name + " " + shape, (x, y-10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 0), 2)

# -------- Masks --------
mask_red = cv2.bitwise_or(
    cv2.inRange(hsv, lower_red1, upper_red1),
    cv2.inRange(hsv, lower_red2, upper_red2)
)

mask_green = cv2.inRange(hsv, lower_green, upper_green)
mask_blue = cv2.inRange(hsv, lower_blue, upper_blue)
mask_yellow = cv2.inRange(hsv, lower_yellow, upper_yellow)

# -------- Detection --------
detect(mask_red, "Red")
detect(mask_green, "Green")
detect(mask_blue, "Blue")
detect(mask_yellow, "Yellow")
```

```
# -------- Show Output (FIXED) --------

print("After Image Processing:")

cv2_imshow(output)
```

Code Explanation:

1. Importing Libraries

import cv2

import numpy as np

from google.colab.patches import cv2_imshow

- `cv2` → OpenCV for image processing
- `numpy` → for arrays (used in masks)
- `cv2_imshow` → special function for displaying images in Google Colab

---

2. Reading and Preparing Image

img = cv2.imread('/input.png')

img = cv2.resize(img, (600, 600))

output = img.copy()

- Reads the image from file
- Resizes it to 600×600
- Creates a copy (`output`) to draw results without modifying original

---

3. Display Original Image

cv2_imshow(output)

Shows the input image before processing

---

4. Convert to HSV Color Space

hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

- OpenCV reads images in **BGR**
- You convert to **HSV (Hue, Saturation, Value)**

Why HSV?
Because it makes **color detection easier and more accurate**

---

5. Define Color Ranges

Example:

lower_red1 = np.array([0, 120, 70])

upper_red1 = np.array([10, 255, 255])

You define **ranges of HSV values** for each color:

- Red (two ranges because red wraps around HSV)
- Green
- Blue
- Yellow

These ranges act like **filters**

---

6. Shape Detection Function

def detect_shape(cnt):

This function identifies shapes using contours:

How it works:

peri = cv2.arcLength(cnt, True)

approx = cv2.approxPolyDP(cnt, 0.04 * peri, True)

- Approximates the contour into simpler polygon

Then:

- 3 points → Triangle
- 4 points → Rectangle
- Otherwise → Circle

---

7. Detection Function (Core Logic)

def detect(mask, color_name):

This is the **main processing function**

Step-by-step:

(a) Noise Removal

mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)

- Removes small holes/noise using morphological operation

(b) Find Contours

contours, _ = cv2.findContours(...)

- Finds boundaries of shapes in the mask

(c) Filter Small Objects

if cv2.contourArea(cnt) > 700:

- Ignores tiny objects (noise)

(d) Detect Shape + Draw

shape = detect_shape(cnt)

x, y, w, h = cv2.boundingRect(cnt)

- Detect shape
- Get bounding box

Then:

cv2.drawContours(...)

cv2.putText(...)

- Draw contour outline
- Label it like `"Red Circle"`

---

8. Creating Masks (Color Detection)

Example:

mask_green = cv2.inRange(hsv, lower_green, upper_green)

- Creates a **binary image (mask)**:
  - White → pixels in that color range
  - Black → everything else

Special case for red:

mask_red = cv2.bitwise_or(...)

- Combines two red ranges

---

9. Run Detection for Each Color

detect(mask_red, "Red")

detect(mask_green, "Green")

detect(mask_blue, "Blue")

detect(mask_yellow, "Yellow")

Each mask is processed separately

---

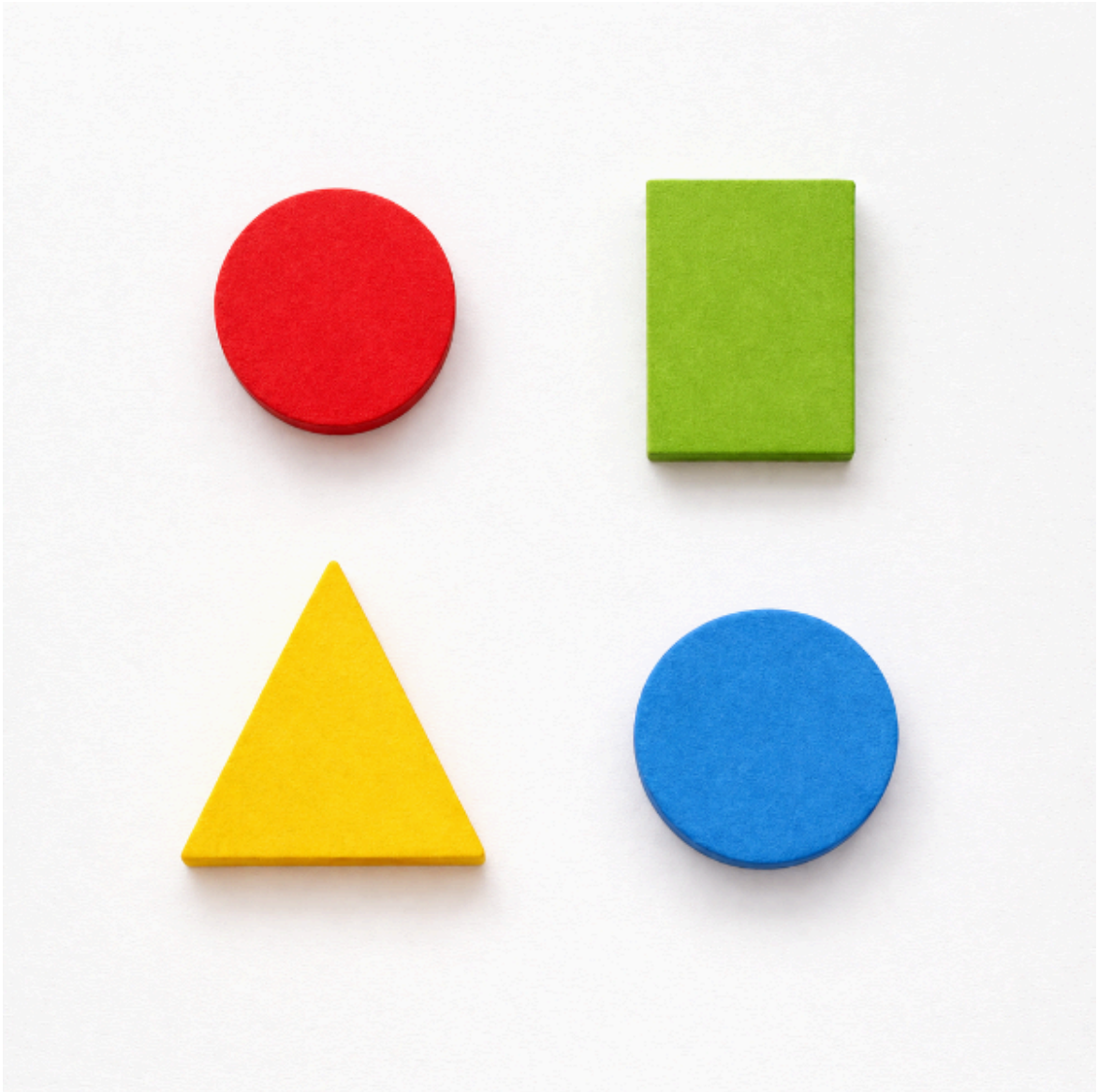
10. Show Final Output

cv2_imshow(output)

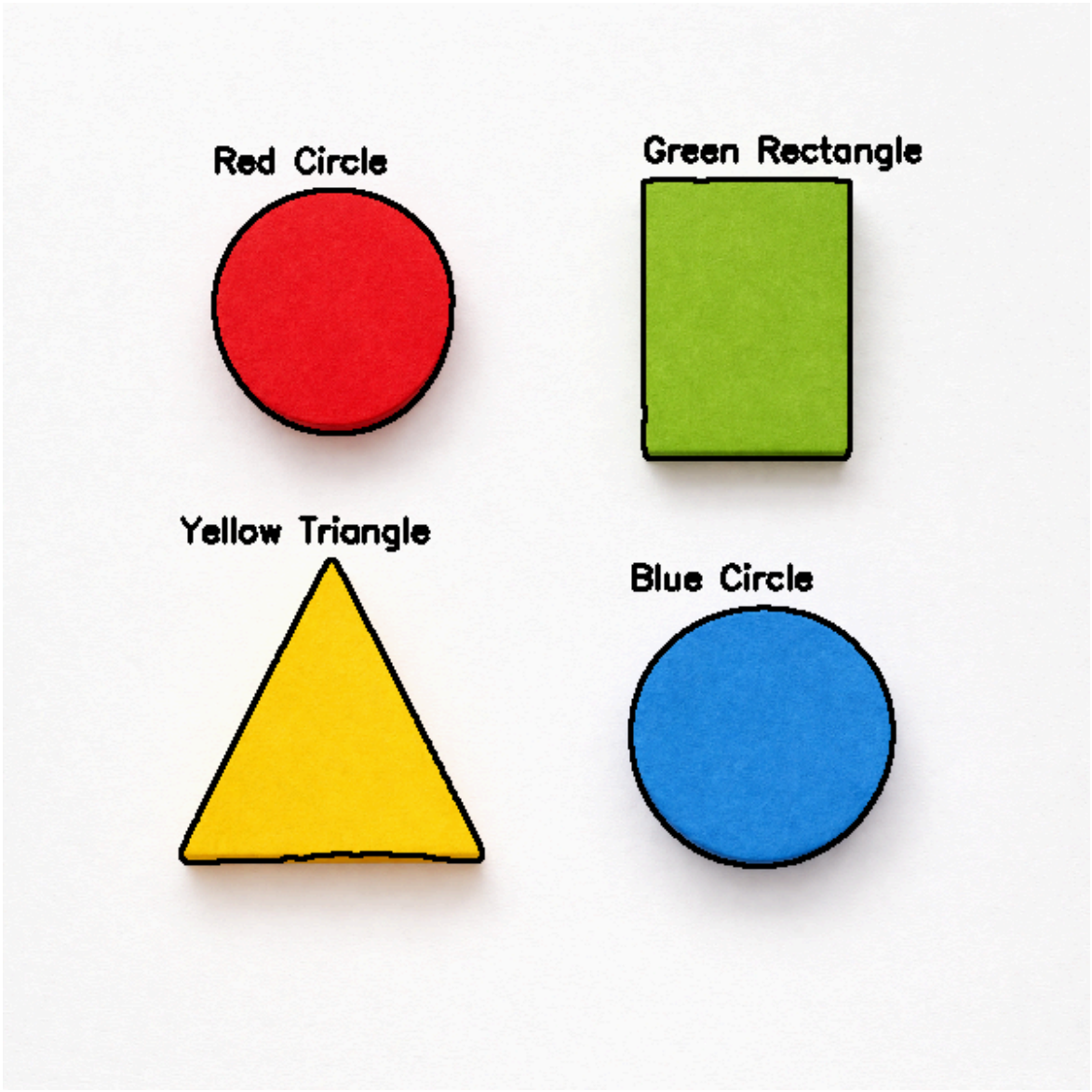
Displays:
Detected shapes
With color labels

Output:

Before Image Processing:



After Image Processing:



Conclusion:

This program demonstrates a practical application of **Computer Vision using OpenCV**, where an image is analyzed to detect both **colors and shapes**.

By converting the image into the HSV color space, the system efficiently isolates specific colors using masking techniques. It then identifies object boundaries through contour detection and determines shapes based on the number of edges. Finally, it labels each detected object with its corresponding **color and shape**, making the output easy to interpret.

Overall, this project highlights how basic image processing techniques—such as **color filtering, contour detection, and shape approximation**—can be combined to build intelligent visual recognition systems. It also serves as a strong foundation for more advanced applications like **object tracking, automation, and real-time vision systems**.